Skylux Calista 3D refactor — scope, data contract & estimate

Calista takes the proposed Climafast data contract: one structure section (even if it's only ever one) plus side openings that dictate both what a side contains and where it sits. The 3D receives the specs and executes — only the strictly-3D model calculation lives here; every structure spec comes from the configurator.

Not in scope

- Handle-value fast paths / optimistic input — the scope here is a refactor of what we already have
- Hotspots
- Dragging
- 2D input (footprint / shape editing)

Handover

Major decisions are commented and explained in-code. During production we can hold high-level sessions walking Skylux through what changed, why it was needed, and how it works. How much of that is needed remains to be seen — ideally Skylux reviews the changes and comes back with questions where they have them, staying largely self-sufficient while we step in only where needed. They hold the initiative.

To-do

Blender work

1. Upgrade the file to Blender 5.2 LTS + Verge3D 4.13
2. Export the environment as a separate GLB
3. Repeatable objects delivered as one entity; clone materials where needed
4. Remove measurements; keep only the primitives for the measurement module
5. Remove all Outdoor/Climafast objects + references (now their own blends)
6. Clean out old materials + test objects (e.g. the general manipulator)
7. Check and clear any old textures
8. Distinct pole meshes — corner (perpendicular faces) vs intermediate (parallel faces) vs wall-adjacent

Logic work

1. Update all Verge3D functions / methods / helpers to the newest (4.13) API — `setMorphFactor` / `changeVis` / `setObjTransform` / `cloneObject` / `makeParent` / … (pairs with Blender #1)

2. `getParameters()` → the sole ingestion point: parse the contract into a single object (`sceneParameters`) that is the only source of configurator data the 3D uses — nothing else reads `scope[...]` directly
3. `structureSection` class — one instance per `_3d_StructureSections[]` entry; array-driven so N sections is data, not code (one today)
    - anchored to its own empty (its handle) — carries the transform; all its objects parent to it
    - positions itself from its own `xPos/yPos/width/depth/heightUnderGutter` , not global width/depth vars
    - owns the roof — width divisions, fillings, rafters, LED, horizontal screen (from `roofOptions`) — plus the gutter
    - roof fillings + rafters keep their interactions and panel openings — operable panels / click-to-open carry over into the section-driven roof
    - existing roof/gutter builders reused, just re-homed onto the section
4. `sideOpening` class — one instance per `_3d_SideOpenings[]` entry — replaces the 5 named `sideSpecs` instances and the 5 copy-pasted per-side loops (front/left/right/rear/standalone)
    - grow it from Climafast's `sideSpecs` — the one that already took the correct steps toward standalone, granular objects; refactoring + optimization still to do
    - anchored to its own empty (its handle) — carries the transform; all its objects parent to it
    - places itself from `sideSpecs.xPos/yPos/rotation/length/height` ; orientation from `rotation` , so the `if side=='Left'…` axis branching disappears
    - owns its `poles[]` , option bays, and gutter
    - reuses the per-opening builders (climaglide / screen / wall / LED), driven per bay
5. Pole placement — each pole seats at its `pole.position` (local X along the side) as its `pole.type` mesh (incl. wall-adjacent); no even-spacing (delete `calculatePoleXpos/Ypos`)
    - base at ground, height morph up to the gutter underside
    - per-pole support from `pole.support` (bracket / foundation)
    - corner poles come from the configurator (2 opposite sides) — no 3D dedup
6. Options read the contract per bay — rewrite the climaglide / vertical sunblind / fixed wall / LED builders to consume each bay's own `option` object instead of per-side globals / `partitions[0]`
    - bays derived from consecutive pole positions + `option.generalSpecs` (replaces `calculateOptionSpacing/CenterPos`)
    - one independent option set per bay, so bays can differ

- two layers per bay — `builtIn` + `surface` slots, each the option sub-schema; builder runs per layer, surface at a depth offset, both may coexist (Calista's biggest divergence from Climafast; formalizes the old `level_1` / `level_2`)
- post accessories (screen/wall rail, plug) are triggered by the bay's option and placed on its poles
- remove the ~5-bay hardcoded ceilings (caches, strides, `[40]`, click list, animation bands, material suffixes)

7. Interaction + animation owned per opening — each `sideOpening` object carries all its own state/vars + handlers for click, hover, and animation (climaglide open, screen/wall); Climafast has the template for this; no fixed-side arrays
8. All measurements go through the measurement module — never hardcoded — driven per opening; replaces the fixed per-side objects + `getTextFromTree`'s 4 labels
9. Adjust the camera framing per opening (index/orientation) instead of the 4 fixed side cases (`tweenCameraToSide`); fix the `front/front/rear` guard
   - use the enhanced zoomfit method for framing (not manual camera positions)
10. Load + toggle the environment from its separate GLB (`appendScene`)
11. Standalone climaglide + wall configurators — keep them in the old file (split off for now); merge onto the shared `sideOpening`/option schema only when big changes are needed there
12. Data robustness as a guiding principle — validate incoming contract data and degrade gracefully; never assume every value is present or well-formed

Estimation

Per-item lines are build hours (engineering + self-test); coordination/tutorializing listed separately.

Blender

#	Item	Hours
1	Blender 5.2 + Verge3D 4.13 upgrade	1
2	environment → separate GLB	1.5–2
3	repeatable objects + clone materials	2.5–3
4	measurements → primitives only	0.5

5	remove Outdoor/Climafast objects + refs	4.5–5
6	clean old materials + test objects	2
7	clear old textures	1
8	distinct pole meshes	4.5–5
	Blender total	**17.5–19.5**

Logic (JS)

#	Item	Hours
1	Verge3D 4.13 API sweep (whole file)	8
2	getParameters single ingestion	4
3	structureSection + re-home roof	12–16
4	sideOpening class	22–28
5	pole placement (position/type/support)	10–12
6	per-bay options + two-layer built/surf	24–32
7	interaction + animation per opening	11–12
8	measurements via the module	7–8
9	camera framing per opening	4
10	environment from separate GLB	1.5–2

11	standalone climaglide/wall split-off	—
12	data-robustness pass	4
	Logic total	**107.5–130**

Summary

Item	Hours
Build combined	~125–150 h
Coordination + 3D-project tutorializing	3–5 days (24–40 h)
Total	**~149–190 h**

Data contract

High-level, adapted from Climafast rev 3 (★ = Calista delta).

```
scope
├ general  ── reused from Climafast ──
│    ├ _3d_screenshotName          string  – screenshot PNG filename
│    ├ _3d_designStyle             string  – Calista is the only style; keep the field, will likely be fixed
│    ├ _3d_conservatoryStructure   string  – "constructionTypeFreeStanding" = freestanding; else wall-adjacent
│    ├ _3d_widthTotal              int mm  – overall width
│    ├ _3d_depthTotal              int mm  – overall depth
│    ├ _3d_walls                   bool[4] – wall present, per side
│    └ _3d_colourOutside           hex     – outside frame colour
│
├ _3d_StructureSections[]          (one today; array kept for future N)
│    └ section
│         ├ width, depth           int mm  – section footprint
│         ├ xPos, yPos             int mm  – section bounding-box centre
│         ├ heightUnderGutter      int mm  – height under the gutter (flat)
│         ├ sectionName            string  – internal name (may be empty)
│         ├ adjacentSections       [left,right] string|null – for redundancy; [null,null] fine for now
│         └ roofOptions
│              ├ horizontalScreen  { enabled:bool · type: zipper|non-zipper|bottom-zipper · colorSet:[3 hex] }
│              ├ roofFillings[]    { position:int mm (edge→centre) · width:int mm · type: PSB|PO|P|SPW|SPA|SPT|G }
│              └ rafters[]         { position:int mm (edge→centre) · ledSpot:[int mm]|null · ledSpotColor:hex
│                                     · ledStrip:bool · ledStripColor:hex }
│
└ _3d_SideOpenings[]
     └ sideOpening
          ├ sideSpecs
          │    ├ xPos, yPos        int mm  – side start point
```

```
          │    ├─ rotation          int deg — absolute (front 270 · rear 90 · left 0 · right 180)
          │    ├─ length, height    int mm  — side extent
          │    ├─ poleAmount        int     — total poles incl. edges (derived from poles[])
          │    ├─ gutter            null    — redundant for now
          │    ├─ wallAttached      null    — redundant for now
          │    └─ sections          null    — redundant for now
          │
          ├─ poles[]
          │    ├─ type              string  — corner | intermediate | wall-adjacent ★ (naming TBD w/ Skylux)
          │    ├─ position          int mm  — local X, pole centre
          │    ├─ length,width,depth int mm — pole dims
          │    ├─ index             int     — configurator index (3D doesn't rely on it)
          │    └─ ★ support         bracket | bracket_under | foundation | null — footing per pole
          │
          └─ options[]              (one per gap between two poles)
               ├─ generalSpecs      { position:int mm (local X) · length:int mm (= pole gap) · height:int mm (up to gutter) }
               ├─ ★ builtIn         «optionSet» | null  — integrated layer
               └─ ★ surface         «optionSet» | null  — surface-mounted layer

«optionSet»  (shared per-layer shape)
     ├─ climaglide        { panelAmount:int · openingDirection: left|right|middle · handle:bool
     │                      · sideProfile:bool · foundationProfile:bool · draftProfile: Alu|Pvc|null
     │                      · glass: clear|opal|null } | null
     ├─ verticalSunblind  { type: axiom100|axiom130|element · screenColor:[3 hex] · solar:bool
     │                      · windowType: no-window|partial|full · windowWidth·windowHeight·windowGroundHeight:int mm } | null
     ├─ fixedWall         { partitions[]: { width·height:int mm
     │                      · fillType: none|woodInside|woodOutside|woodBothSides|sandwichPanelAnthracite|sandwichPanelWhite|glass } } | null
     ├─ ledStrip          bool
     └─ (post accessory derived — chosen option triggers the matching rail/plug on the poles)
```

Need from Skylux

1. Gutter: one continuous profile (poles bolted underneath) or separate sections between poles? Leaning continuous — confirm the real product matches.
2. Pole geometry/types — confirm corner vs intermediate vs wall-adjacent (which meshes/types exist)

Structure skeleton

Commented scaffold of the refactored product logic — structure only (classes, methods, call order, responsibilities), no real bodies. Each block maps to a Logic to-do; an engineer can fill it top-down.

```
// ═══════════════════════════════════════════════════════════════
//  SKYLUX CALISTA — data-driven refactor (SKELETON — structure only)
//
```

```
4  //  Flow:  scope[]  →  getParameters()  →  sceneParameters (one object)
5  //         →  generic structureSection + sideOpening instances execute the specs.
6  //
7  //  Verge3D 4.13 puzzle helpers only (setObjTransform / setMorphFactor / changeVis /
8  //  cloneObject / makeParent / appendScene). No raw three.js.  [Logic #1]
9  //  ═══════════════════════════════════════════════════════════════════════
10
11 let sceneParameters = {};
12
13 // Instances are cached by index and REUSED across rebuilds (grown on demand).
14 const structureSectionCache = [];
15 const sideOpeningCache = [];
16
17
18 //  ─────────────────────────────────────────────────────────────────────
19 //  1 · INGESTION — the only place scope[] is read  [Logic #2, #12]
20 //  ─────────────────────────────────────────────────────────────────────
21 /**
22  * Parse the raw configurator scope into ONE sceneParameters object.
23  * Every field is read guarded (validate → default) so a single malformed
24  * value can never abort the whole build. Nothing else reads scope[] directly.
25  */
26 function getParameters() {
27     // sceneParameters.general           ← widthTotal / depthTotal / walls / colourOutside / designStyle …
28     // sceneParameters.structureSections ← _3d_StructureSections[] (guarded, default [])
29     // sceneParameters.sideOpenings      ← _3d_SideOpenings[] (guarded, default [])
30     // sceneParameters.render            ← ssr / environment flags
31 }
32
33
34 //  ─────────────────────────────────────────────────────────────────────
35 //  2 · SCENE ORCHESTRATION
36 //  ─────────────────────────────────────────────────────────────────────
37 function enableScene() {
38     // clearScene();
39     // getParameters();
40     // enableScene3D();
41     // renderFrame();                    ← single render point
42 }
43
44 function enableScene3D() {
45     // one generic instance per spec, cached by index, reused across rebuilds:
46     sceneParameters.structureSections.forEach((spec, index) => {
47         // structureSectionCache[index] ??= new structureSection();
48         // structureSectionCache[index].enable(spec, index);
49     });
50     sceneParameters.sideOpenings.forEach((spec, index) => {
51         // sideOpeningCache[index] ??= new sideOpening();
52         // sideOpeningCache[index].enable(spec, index);
53     });
```

```
    // hideLeftovers(structureSectionCache,
sceneParameters.structureSections.length);
    // hideLeftovers(sideOpeningCache,
sceneParameters.sideOpenings.length);
    // enableEnvironment();               ← appended from its own
GLB, toggled here   [Logic #10]
}


//
──────────────────────────────────────────────────────────────────────
──
//  3 · STRUCTURE SECTION — one per _3d_StructureSections[] entry
[Logic #3]
//      Array-driven (one today, N later). Owns roof + crown.
//
──────────────────────────────────────────────────────────────────────
──
class structureSection {
    constructor() {
        // this.handle = cloneObject(<empty template>);   ← anchor;
every object parents to it
    }
    enable(spec, index) {
        // position this.handle from spec.xPos / spec.yPos (children
stay local)
        // this.enableCrown(spec);        ← continuous crown, all 4
sides
        // this.enableRoof(spec.roofOptions);
    }
    enableCrown(spec)        { /* place + morph the crown to the
section footprint */ }
    enableRoof(roofOptions)  {
        // this.enableRoofDivisions(...);
        // this.enableRoofFillings(roofOptions.roofFillings);  ←
operable panels keep click-to-open  [Logic #3]
        // this.enableRafters(roofOptions.rafters);            ←
incl. led spots / led strip
        // this.enableHorizontalScreen(roofOptions.horizontalScreen);
    }
    // roof/crown builders reused from the current file, re-homed
onto this.handle
    enableRoofDivisions()    { }
    enableRoofFillings()     { }
    enableRafters()          { }
    enableHorizontalScreen() { }
}


//
──────────────────────────────────────────────────────────────────────
──
//  4 · SIDE OPENING — one per _3d_SideOpenings[] entry
[Logic #4-8]
//      Grown from Climafast's sideSpecs. Replaces the 5 named specs
//      + the 5 copy-pasted per-side loops. Owns its OWN interaction/
//      animation state — no shared fixed-side arrays.
//
──────────────────────────────────────────────────────────────────────
──
class sideOpening {
    constructor() {
        // this.handle = cloneObject(<empty template>);
        // per-opening caches + interaction/animation state (poles,
bays,
        // climaglide open %, screen open %, …) live on the instance.
    }
    enable(spec, index) {
        // place this.handle from spec.sideSpecs.xPos / yPos /
rotation
        //   → orientation comes from rotation, so no `if side ==
'Left'` branching
```

```
        // this.enablePoles(spec.poles);
        // this.enableBays(spec.options);
        // this.registerInteractions();   ← bound once, on first build
    }

    // ── poles ──────────────────────────────────────────────── [Logic #5]
    enablePoles(poles) {
        // for each pole:
        //   seat at pole.position (local X) as its pole.type mesh (corner / intermediate / wall-adjacent)
        //   height morph up to the crown underside
        //   footing from pole.support (bracket / foundation / none)
    }

    // ── option bays ────────────────────────────────────────── [Logic #6]
    enableBays(options) {
        // for each bay (the gap between two consecutive poles):
        //   this.enableBay(bay);
    }
    enableBay(bay) {
        // bay.generalSpecs → the bay's local position / length / height
        // two independent layers, each an optionSet or null:
        // this.enableLayer(bay.builtIn, 'builtIn');   ← integrated
        // this.enableLayer(bay.surface, 'surface');   ← surface-mounted, offset outward in depth
    }
    enableLayer(optionSet, layer) {
        // if (!optionSet) return;
        // if (optionSet.climaglide) this.enableClimaglide(optionSet.climaglide, layer);
        // if (optionSet.verticalSunblind) this.enableVerticalSunblind(optionSet.verticalSunblind, layer);
        // if (optionSet.fixedWall) this.enableFixedWall(optionSet.fixedWall, layer);
        // if (optionSet.ledStrip) this.enableLedStrip(layer);
        // chosen option also triggers its post accessory (rail / plug) on the bay's poles
    }

    // ── option builders (ported from the existing per-side builders, now per bay) ──
    enableClimaglide()       { }
    enableVerticalSunblind() { }
    enableFixedWall()        { }
    enableLedStrip()         { }

    // ── interaction + animation, owned by THIS opening ──────── [Logic #7]
    registerInteractions()   { /* click / hover, bound once, keyed to this opening's objects */ }
    animateClimaglide()      { }
    animateScreen()          { }

    // ── measurements for this opening, via the measurement module ─ [Logic #8]
    enableMeasurements()     { }
}


//──────────────────────────────────────────────────────────────────────
//  5 · CAMERA — frame per opening/index via zoomfit                [Logic #9]
//──────────────────────────────────────────────────────────────────────
```

```
function frameOpening(index) {
    // zoomfit onto sideOpeningCache[index].handle (no fixed left/right/front/rear cases)
}


//  ──────────────────────────────────────────────────────────────────────────
//  NOTE: the standalone climaglide + wall configurators stay in the OLD file
//  for now; they merge onto this shared sideOpening/option schema later.   [Logic #11]
//  ──────────────────────────────────────────────────────────────────────────

```